

Lab - Containerizing an Application

To Begin with the Lab

Summary

This lab demonstrates how to containerize a simple ASP.NET Core Razor Pages application using Docker. A multi-stage Dockerfile is created to build the application with the .NET SDK image and run it using the ASP.NET runtime image, producing a lightweight and portable Docker image.

Understanding

Containerizing an application means packaging the application along with its runtime, dependencies, and configuration into a Docker image. This image can then be run consistently on any system that has Docker installed.

A **Dockerfile** contains the instructions required to build the image. In this lab, a **multi-stage build** is used:

- **Build Stage** uses the .NET SDK image to restore packages, build, and publish the application.
- **Runtime Stage** uses the smaller ASP.NET Runtime image to run the published application.

This approach reduces the final image size because only the published application is included in the runtime image.

Example:

Suppose you create an ASP.NET Core application named **DockerApp**.

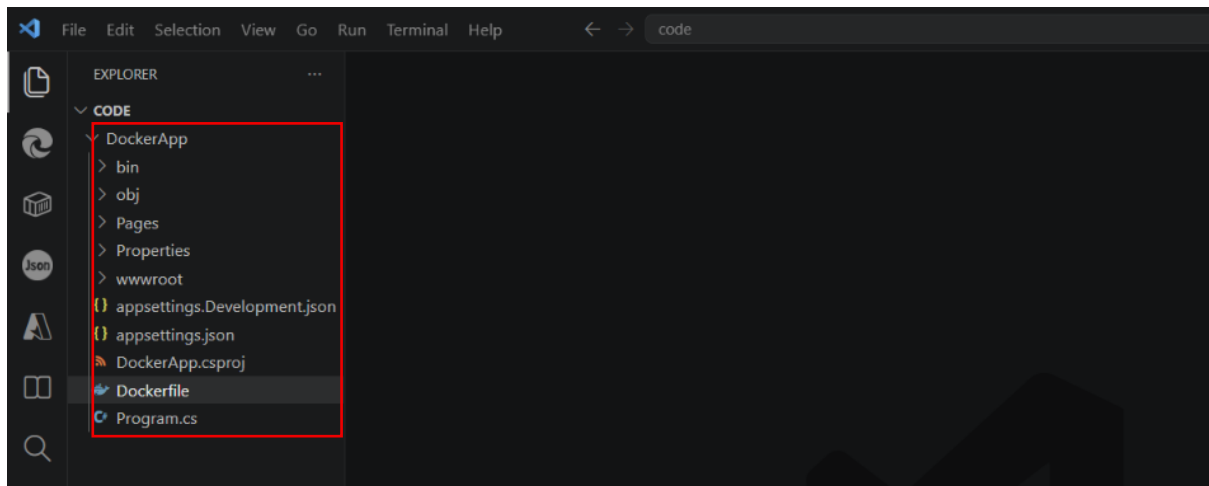
- **Build Stage:**
 - Uses the .NET SDK image.
 - Runs dotnet restore.
 - Runs dotnet publish.
 - Produces the published application files.
- **Runtime Stage:**
 - Uses the ASP.NET Runtime image.
 - Copies the published files.
 - Exposes port **8080**.
 - Starts the application using dotnet DockerApp.dll.

The resulting Docker image can be run on any machine without installing the .NET SDK.

Lab Steps

Step 1: Create a New ASP.NET Core Project

- Open Visual Studio Code.
- Open an empty project folder.
- Create a new ASP.NET Core Razor Pages application.
- Give the project a suitable name (for example, **DockerApp**).

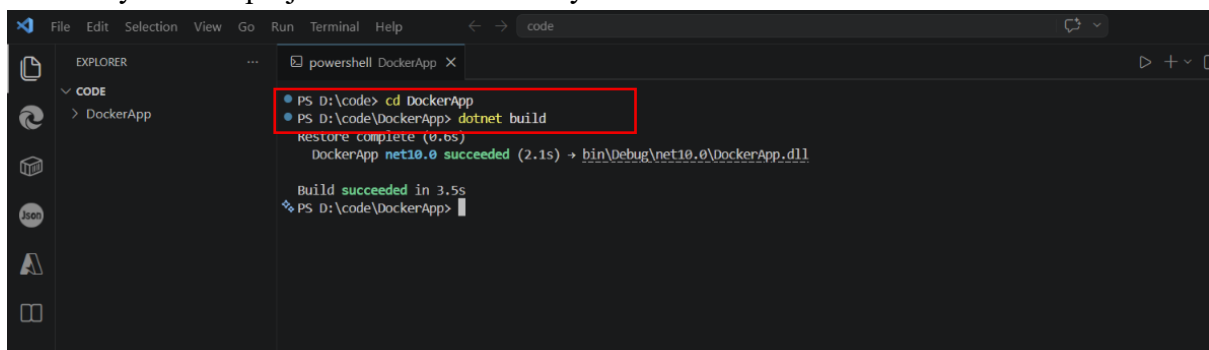


Step 2: Build the Application

- Open the **terminal**.
- Navigate to the project folder.
- Run:

dotnet build

- Verify that the project builds successfully.



Step 3: Run the Application Locally

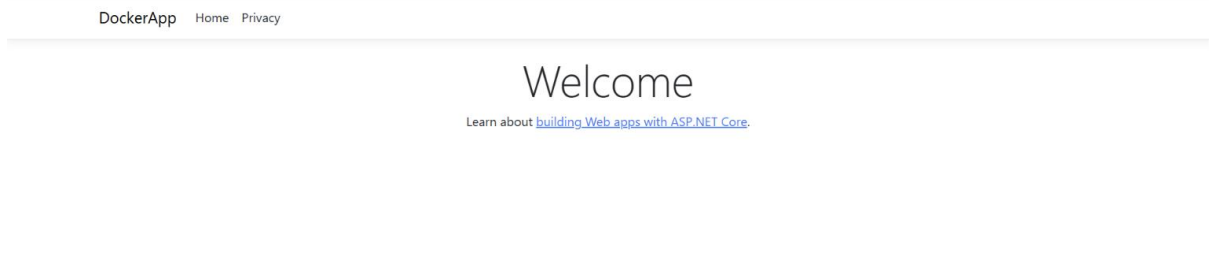
- Execute:

dotnet run

- Copy the **local host Url**
- Open the displayed localhost URL.
- Verify that the Razor Pages application opens successfully.

```
powershell DockerApp X
PS D:\code> cd DockerApp
PS D:\code\DockerApp> dotnet build
Restore complete (0.6s)
DockerApp net10.0 succeeded (2.1s) -> bin\Debug\net10.0\DockerApp.dll

Build succeeded in 3.5s
PS D:\code\DockerApp> dotnet run
Using launch settings from D:\code\DockerApp\Properties\launchSettings.json...
Building...
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5207
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: D:\code\DockerApp
info: Microsoft.Hosting.Lifetime[0]
      Application is shutting down...
PS D:\code\DockerApp>
```



- Stop the application using **Ctrl + C**.

Step 4: Create a Dockerfile

- Inside the project folder, create a new file named: Dockerfile
- This file contains the instructions for building the Docker image.
- Paste this code in **Dockerfile**.

```
# ---- Build stage ----
FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
WORKDIR /src

# copy csproj and restore first (better caching)
COPY *.csproj ./
RUN dotnet restore

# copy the rest and publish
COPY . ./
RUN dotnet publish -c Release -o /app/publish

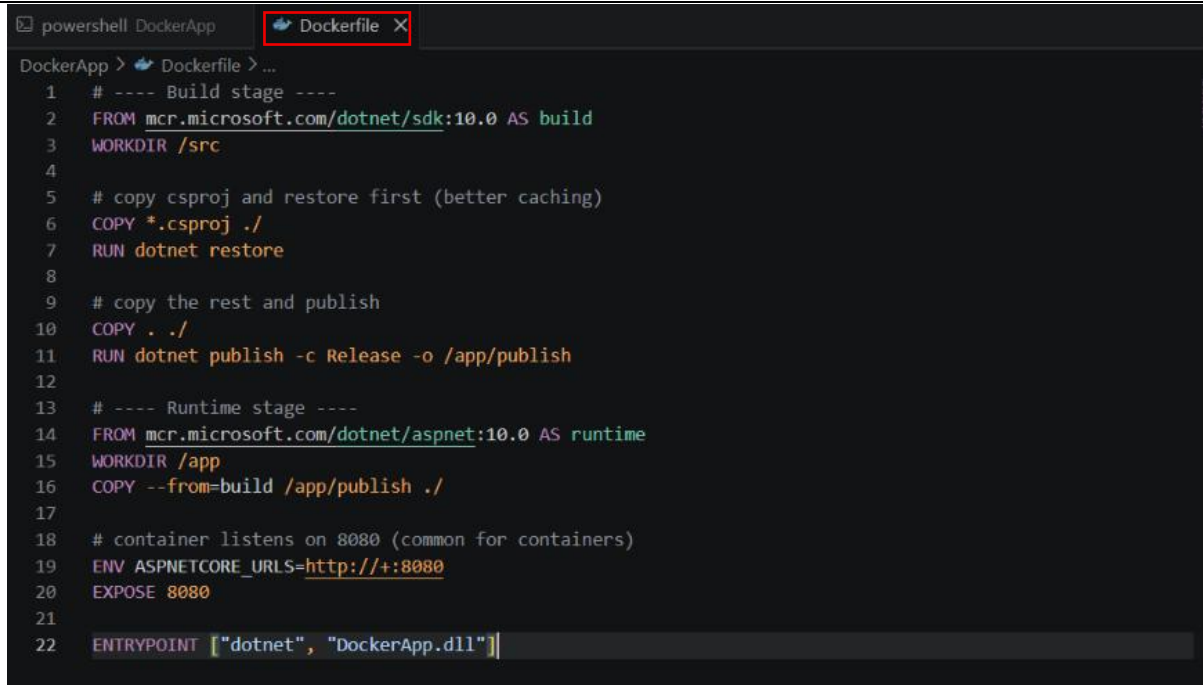
# ---- Runtime stage ----
FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
WORKDIR /app
COPY --from=build /app/publish ./
```

container listens on 8080 (common for containers)

ENV ASPNETCORE_URLS=http://+:8080

EXPOSE 8080

ENTRYPOINT ["dotnet", "DockerApp.dll"]



```
powershell DockerApp Dockerfile X
DockerApp > Dockerfile > ...
1 # ---- Build stage ----
2 FROM mcr.microsoft.com/dotnet/sdk:10.0 AS build
3 WORKDIR /src
4
5 # copy csproj and restore first (better caching)
6 COPY *.csproj ./
7 RUN dotnet restore
8
9 # copy the rest and publish
10 COPY . ./
11 RUN dotnet publish -c Release -o /app/publish
12
13 # ---- Runtime stage ----
14 FROM mcr.microsoft.com/dotnet/aspnet:10.0 AS runtime
15 WORKDIR /app
16 COPY --from=build /app/publish ./
17
18 # container listens on 8080 (common for containers)
19 ENV ASPNETCORE_URLS=http://+:8080
20 EXPOSE 8080
21
22 ENTRYPOINT ["dotnet", "DockerApp.dll"]
```